Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce Department of Information Technology
(B.Sc.I.T Semester V)
Full Stack development MERN Practical
Practical– IX

Roll No.: T055	Name: Achal Yadav
Class: TYIT	Batch: 02
Date of Assignment: : 30/09/2026	Date/Time of Submission: 30/09/2026

Practical 9: User Data Management

Aim

To create a User Data Management application using **Express, MongoDB and React** for adding, retrieving, updating and deleting user records.

Part A: Create MongoDB Database

Steps:

1. Open MongoDB Compass.
2. Connect to `mongodb://127.0.0.1:27017`.
3. Create database `UserDB`.
4. Create collection `users`.
5. Insert sample user records.
6. Use `find()` to display the records.

Part B: Create Backend

Steps:

1. Create a folder named `practical9`.
2. Open the folder in VS Code.
3. Run `npm init -y`.
4. Install `mongoose`, `express` and `cors`.
5. Create `server.js`.
6. Connect Express with MongoDB `UserDB`.
7. Create GET, POST, PUT and DELETE APIs.
8. Start the server using `node server.js`.

Part C: Create Frontend

Steps:

1. Create a React project using Vite.
2. Install Axios.
3. Open `src/App.jsx`.
4. Create a form for entering user details.
5. Use Axios to communicate with the Express API.
6. Display users in a table.
7. Add Edit and Delete buttons.

Part D: Execution

Backend:

node server.js

Frontend:

npm run dev

The backend and frontend are executed in **two separate terminals**.

CRUD Operations

Operation	HTTP Method	API
Retrieve users	GET	`/users`
Add user	POST	`/users`
Update user	PUT	`/users/:id`
Delete user	DELETE	`/users/:id`

Output

Server,js:-

```
const express = require("express");

const mongoose = require("mongoose");

const cors = require("cors");
```

```
const app = express();

const PORT = 3000;


// Middleware

app.use(cors());

app.use(express.json());


// MongoDB Connection

mongoose

  .connect("mongodb://127.0.0.1:27017/UserDB")

  .then(() => {

    console.log("MongoDB Connected Successfully");

  })

  .catch((error) => {

    console.log("MongoDB Connection Failed");

    console.log(error);

  });


// GET - Retrieve Users

app.get("/users", async (req, res) => {

  try {

    const users = await mongoose.connection.db

      .collection("users")

      .find()
```

```
      .toArray();

    res.json(users);
  } catch (error) {
    res.status(500).json({
      message: "Error fetching users",
    });
  }
});

// POST - Add User
app.post("/users", async (req, res) => {
  try {
    const user = req.body;

    await mongoose.connection.db
      .collection("users")
      .insertOne(user);

    res.json({
      message: "User added successfully",
    });
  } catch (error) {
    res.status(500).json({
      message: "Error adding user",
```

```
    });
  }
});


// PUT - Update User
app.put("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id);
    const updatedUser = req.body;


    await mongoose.connection.db
      .collection("users")
      .updateOne(
        { _id: id },
        {
          $set: {
            name: updatedUser.name,
            email: updatedUser.email,
            city: updatedUser.city,
          },
        }
      );


    res.json({
      message: "User updated successfully",
```

```
    });
  } catch (error) {
    res.status(500).json({
      message: "Error updating user",
    });
  }
});


// DELETE - Delete User
app.delete("/users/:id", async (req, res) => {
  try {
    const id = new mongoose.Types.ObjectId(req.params.id);


    await mongoose.connection.db
      .collection("users")
      .deleteOne({
        _id: id,
      });


    res.json({
      message: "User deleted successfully",
    });
  } catch (error) {
    res.status(500).json({
      message: "Error deleting user",
```

```
    });
  }
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

App.jsx:-

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  // Store all users
  const [users, setUsers] = useState([]);

  // Form data
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [city, setCity] = useState("");

  // Store ID of user being edited
  const [editId, setEditId] = useState(null);
```

```
// Get Users from MongoDB
const getUsers = () => {
  axios
    .get("http://localhost:3000/users")
    .then((response) => {
      setUsers(response.data);
    })
    .catch((error) => {
      console.log(error);
    });
};

// Load users when page starts
useEffect(() => {
  getUsers();
}, []);

// Add or Update User
const saveUser = () => {
  const user = {
    name: name,
    email: email,
    city: city,
  };
```

```
// UPDATE existing user
if (editId) {
  axios
    .put(`http://localhost:3000/users/${editId}`, user)
    .then((response) => {
      alert(response.data.message);
      clearForm();
      getUsers();
    })
    .catch((error) => {
      console.log(error);
    });
}

// ADD new user
else {
  axios
    .post("http://localhost:3000/users", user)
    .then((response) => {
      alert(response.data.message);
      clearForm();
      getUsers();
    })
    .catch((error) => {
      console.log(error);
```

```
      });
  }
};

// Edit User
const editUser = (user) => {
  setEditId(user._id);
  setName(user.name);
  setEmail(user.email);
  setCity(user.city);
};

// Delete User
const deleteUser = (id) => {
  if (window.confirm("Are you sure you want to delete this user?")) {
    axios
      .delete(`http://localhost:3000/users/${id}`)
      .then((response) => {
        alert(response.data.message);
        getUsers();
      })
      .catch((error) => {
        console.log(error);
      });
  }
```

```
};


// Clear Form
const clearForm = () => {
  setName("");
  setEmail("");
  setCity("");
  setEditId(null);
};


return (
  <div>
    <h1>User Management System</h1>


    {/* User Form */}
    <h2>{editId ? "Update User" : "Add User"}</h2>


    <input
      type="text"
      placeholder="Enter Name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />


    <br />
```

```
      <br />


      <input
        type="email"
        placeholder="Enter Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />


      <br />
      <br />


      <input
        type="text"
        placeholder="Enter City"
        value={city}
        onChange={(e) => setCity(e.target.value)}
      />


      <br />
      <br />


      <button onClick={saveUser}>
        {editId ? "Update User" : "Add User"}
      </button>
```

```jsx
{editId && (
  <button onClick={clearForm}>
    Cancel
  </button>
)}

<hr />

{/* Display Users */}
<h2>User List</h2>

<table border="1">
  <thead>
    <tr>
      <th>Name</th>
      <th>Email</th>
      <th>City</th>
      <th>Actions</th>
    </tr>
  </thead>

  <tbody>
    {users.map((user) => (
      <tr key={user._id}>
```

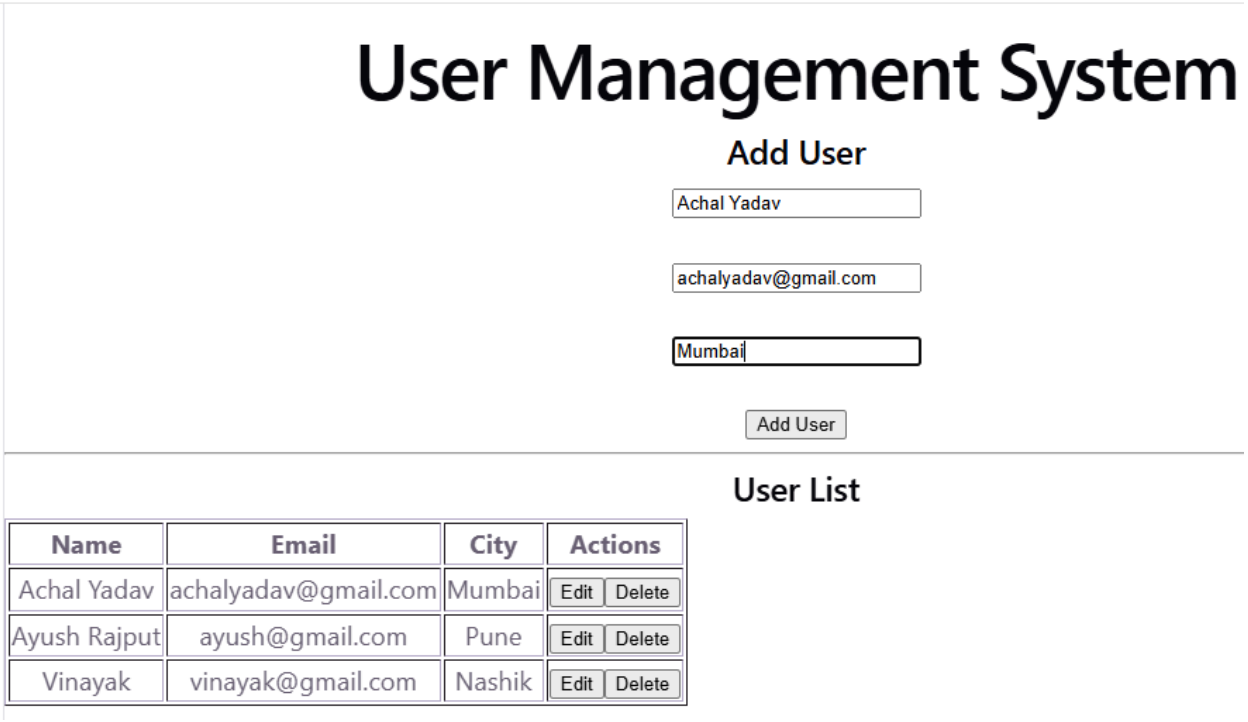
```
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.city}</td>

              <td>
                <button onClick={() => editUser(user)}>
                  Edit
                </button>

                <button onClick={() => deleteUser(user._id)}>
                  Delete
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

Output:-



Result

The User Data Management application was successfully created using **React, Express, MongoDB and Axios**, implementing complete CRUD operations.